

LogPulse AI - Incident Report

Generated : 2026-06-01 12:44:17 UTC

Trace ID : 1e7e199b-19a5-4261-af4d-7127c770cd17

1 - Incident Summary

Incident ID : 1e7e199b
Category : Memory
Severity : SeverityLevel.FATAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

2026-01-15T22:14:07Z FATAL kubelet: Pod worker-api-7d9f8b-xk2q OOMKilled. Container exceeded memory limit of 256Mi. Exit code 137. Restart count: 14. Node: worker-1. Namespace: production.

3 - Classification

Category : Memory
Severity : SeverityLevel.FATAL
Summary : Kubernetes pod killed due to OOMKilled container exceeding its memory limit.
Source : Kubernetes

4 - Diagnosis

Root Cause : The container's memory limit of 256Mi is insufficient for its operational requirements, causing it to be repeatedly OOMKilled by Kubernetes.
Confidence : 95%
Reasoning : The raw log explicitly states 'OOMKilled. Container exceeded memory limit of 256Mi' with a high restart count, and the retrieved playbook is specifically for 'Pod OOMKilled: Insufficient Memory Limit' with a resolution to increase the memory limit.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Pod OOMKilled: Insufficient Memory Limit' (RAG confidence=0.8185). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

1. Verify the `OOMKilled` status using `kubectl get pod <pod-name> -o yaml`.
2. Check container memory usage metrics (e.g., Prometheus/Grafana) to identify peak memory consumption.
3. Analyze application logs for clues about memory spikes or growth patterns.
4. Increase the `resources.limits.memory` for the affected container in its Deployment/StatefulSet manifest.
5. Roll out the updated deployment using `kubectl apply -f <manifest-file>` or `kubectl rollout restart deployment/<deployment-name>`.
6. name: my-app

Commands:

```
$ kubectl get pod worker-api-7d9f8b-xk2q -o yaml
$ kubectl apply -f <manifest-file>
$ kubectl rollout restart deployment/worker-api
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.WARNING**

Safe Commands (2):

```
[OK] kubectl get pod worker-api-7d9f8b-xk2q -o yaml
[OK] kubectl apply -f <manifest-file>
```

Flagged Commands (1):

```
[!] kubectl rollout restart deployment/worker-api [kubectl\s+rollout\s+restart]
```

7 - Operator Decision

Operator rejected -- no execution.

8 - Historical Context

Times Seen : 1

First occurrence -- no prior incidents.

9 - Incident Timeline

```
[09:37:56] INCIDENT_RECEIVED -> Incident received (trace_id=1e7e199b...)
[09:37:56] CACHE_MISS -> Cache miss -- routing to classifier
[09:38:06] INCIDENT_CLASSIFIED -> Memory / Fatal
[09:38:07] RAG_RETRIEVED -> Playbook retrieved (filtered) -- Memory conf=0.82
[09:38:18] ROOT_CAUSE_GENERATED -> The container's memory limit of 256Mi is insufficient for its
operational requirements, causing it to be repeatedly OOMK
[09:39:14] SOLUTION_GENERATED -> 6 step(s) from playbook
[09:39:14] RISK_ASSESSED -> warning -- 1 flagged
[09:44:14] WORKFLOW_COMPLETED -> Completed in 378733ms
```